

✿ I. Rappel : La Nature de l'Objet en JavaScript

◆ 1. Qu'est-ce qu'un objet ?

En JavaScript, un **objet** est une **collection de paires clé/valeur**.

Un objet contient :

- **des propriétés** → données
- **des méthodes** → fonctions stockées dans l'objet

✓ Exemple d'objet :

```
js

let personne = {
  nom: "Diallo",
  age: 25,
  parler: function () {
    console.log("Bonjour !");
  }
};

console.log(personne.nom);    // Affiche : Diallo
personne.parler();           // Appelle la méthode
```

◆ 2. Accéder aux données d'un objet

✓ Deux syntaxes existent :

▶ Notation pointée

```
js

console.log(personne.nom);
```

▶ Notation crochet

js

```
console.log(personne["age"]);
```

💡 La notation crochet est obligatoire si :

- la clé contient un espace → obj["nom complet"]

Ex :

js

```
let obj = {  
  "nom complet": "Diallo",  
  "âge-personne": 25,  
  "1erElement": "test"  
};  
  
console.log(obj["nom complet"]);  
console.log(obj["âge-personne"]);  
console.log(obj["1erElement"]);
```

- la clé est dans une variable → obj[clé]

Ex :

js

```
let cle = "nom";  
let personne = { nom: "Diallo" };  
  
console.log(personne[cle]); // ✔ fonctionne  
console.log(personne.cle); // ✗ ne fonctionne pas (cherche la propriété "cle")
```

◆ 3. Méthodes d'inspection Object.keys / values / entries

✓ Exemple pratique :

```
js

let voiture = {
  marque: "Toyota",
  modele: "Corolla",
  annee: 2020
};

console.log(Object.keys(voiture));
// ["marque", "modele", "annee"]

console.log(Object.values(voiture));
// ["Toyota", "Corolla", 2020]

console.log(Object.entries(voiture));
// [["marque","Toyota"], ["modele","Corolla"], ["annee",2020]]
```

Très utile pour :

- faire des boucles

Ex:

```
js

let utilisateur = {
  nom: "Diallo",
  age: 25,
  pays: "Guinée"
};

for (let [cle, valeur] of Object.entries(utilisateur)) {
  console.log(cle + " : " + valeur);
}
```

Résultat :

yaml

```
nom : Diallo
age : 25
pays : Guinée
```

- déboguer

Ex:

js

```
let produit = {
  nom: "Chaussure",
  prix: 45000
  // stock oublié !
};

let keys = Object.keys(produit);

if (!keys.includes("stock")) {
  console.warn("⚠ Attention : la propriété 'stock' manque !");
}

console.log("Objets reçus :", Object.entries(produit));
```

Résultat :

lua

```
⚠ Attention : la propriété 'stock' manque !
Objets reçus : [["nom","Chaussure"],["prix",45000]]
```

- analyser un objet rapidement

Ex:

```
js

let notes = {
  math: 14,
  physique: 12,
  informatique: 18
};

let valeurs = Object.values(notes);

let total = valeurs.reduce((somme, v) => somme + v, 0);
let moyenne = total / valeurs.length;

console.log("Somme =", total);
console.log("Moyenne =", moyenne);
```

Résultat :

```
ini

Somme = 44
Moyenne = 14.67
```

🏠 II. La POO en JavaScript

◆ 1. Classe et Instance

✓ Une classe = un modèle

✓ Une instance = un objet créé à partir de ce modèle

🎯 Exemple simple :

```
js

class Animal {
  constructor(nom, type) {
    this.nom = nom;
    this.type = type;
  }

  crier() {
    console.log(this.nom + " fait un bruit !");
  }
}

let chien = new Animal("Rex", "Chien"); // instance
chien.crier();
```

1) Déclaration d'une classe

```
js

class Animal {
```

Ici, on déclare une classe appelée Animal.

Une classe est comme un plan ou un modèle pour créer des objets.

2) Le constructeur

```
js

  constructor(nom, type) {
    this.nom = nom;
    this.type = type;
  }
```

- Le constructeur est une fonction spéciale qui s'exécute automatiquement quand tu crées un nouvel objet avec new.
- Les paramètres nom et type sont fournis lors de la création de l'objet.
- this.nom et this.type créent des propriétés dans l'objet.

Exemple :

this.nom = nom veut dire : « la propriété *nom* de l'objet prend la valeur du paramètre *nom* ».

3) Une méthode dans la classe

```
js

crier() {
  console.log(this.nom + " de type " + this.type + " fait un bruit !");
}
```

crier() est une méthode (une fonction) de la classe.

Elle peut être utilisée par toutes les instances de Animal.

4) Création d'une instance

```
js

let chien = new Animal("Rex", "Chien");
```

Quand tu écris new Animal(...), JavaScript :

1. crée un nouvel objet vide,
2. exécute le constructeur avec les paramètres "Rex" et "Chien",
3. retourne l'objet initialisé.

On obtient donc un objet :

```
js

{
  nom: "Rex",
  type: "Chien"
}
```

5) Appel d'une méthode

```
js  
  
chien.crier();
```

Cela appelle la méthode `crier()` sur l'objet `chien`, donc dans la console on verra :

```
bash  
  
Rex de type Chien fait un bruit !
```

♦ 2. Les 4 Piliers de la POO

1. Encapsulation

→ Regrouper données + méthodes

→ Protéger certaines données

```
js  
  
class Compte {  
  constructor(solde) {  
    this._solde = solde; // propriété "privée" par convention  
  }  
  
  afficherSolde() {  
    console.log(this._solde + " €");  
  }  
}
```

2. Héritage

→ Une classe peut dériver d'une autre

→ Réutilisation du code


```
js

class Animal {
  crier() { console.log("Un animal crie"); }
}

class Chien extends Animal {
  crier() { console.log("Le chien aboie"); }
}

new Chien().crier(); // Le chien aboie
```

3. Polymorphisme

→ Une méthode peut se comporter différemment selon l'objet qui l'utilise

Exemple : même méthode crier() mais résultat différent pour chaque classe.

4. Abstraction

→ Cacher les détails

→ Montrer seulement ce qui est important

```
js

class Voiture {
  demarrer() {
    this._injecterCarburant();
    this._tournerCle();
    console.log("La voiture démarre");
  }

  _injecterCarburant() {} // détails cachés
  _tournerCle() {}
}
```

Conclusion

**« Le DOM m'a permis de comprendre comment manipuler la page web.
Ensuite, la POO m'aide à structurer le code avec des objets et des classes pour créer des
applications plus propres, plus lisibles et plus évolutives. »**